**~\OneDrive - cross-ING AG\Desktop\M5.py**

```python
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import skew, kurtosis
from pypfopt.efficient_frontier import EfficientFrontier
from pypfopt.discrete_allocation import DiscreteAllocation
from pypfopt import risk_models
from pypfopt import expected_returns

#tickers of the stocks
tickers = ['NVDA', 'XOM', 'CAT', 'MRK']

# Download historical data from Yahoo Finance
data = yf.download(tickers, start="2022-02-01", end="2024-02-01", interval="1d")['Adj Close']

# Calculate daily returns
daily_returns = data.pct_change().dropna()

# Calculate statistics
average_return = daily_returns.mean() * 252  # Annualized
volatility = daily_returns.std() * np.sqrt(252)  # Annualized
skewness = daily_returns.skew()
kurtosis_values = daily_returns.apply(kurtosis)

# Correlation and Covariance matrices
correlation_matrix = daily_returns.corr()
covariance_matrix = daily_returns.cov() * 252  # Annualized

# Create a DataFrame for statistics
statistics_df = pd.DataFrame({
    "Average Return": average_return,
    "Volatility": volatility,
    "Skewness": skewness,
    "Kurtosis": kurtosis_values
})

#correlation and covariance matrices
sns.set(font_scale=2)
sns.heatmap(correlation_matrix.rename_axis(index=None, columns=None), annot=True, cmap="coolwarm", fmt=".2f", annot_kws={'size':25})
plt.title('Correlation Matrix', fontsize=25)
plt.show()

# Plotting the covariance matrix
sns.heatmap(covariance_matrix.rename_axis(index=None, columns=None), annot=True, cmap="viridis", fmt=".2f", annot_kws={'size':25})
plt.title('Covariance Matrix', fontsize=25)
plt.show()

plt.tight_layout()
plt.show()

```

```python
transposed_df = statistics_df.T
formatted_df = transposed_df.applymap(lambda x: f"{x:.4f}")

# Create a figure for the table
fig, ax = plt.subplots(figsize=(10, 4))  # Adjust size as needed
ax.axis('off')


# Adding the table to the plot
table = ax.table(cellText=formatted_df.values,
                 colLabels=formatted_df.columns,
                 rowLabels=formatted_df.index,
                 cellLoc='center',
                 loc='center')

table.auto_set_font_size(False)
table.set_fontsize(15)
table.scale(1, 3)

plt.tight_layout()
plt.show()


# ------------------ OPTIMIZATION ----------------------------------------------------

# Expected returns and sample covariance
mu = expected_returns.mean_historical_return(data)
S = risk_models.sample_cov(data)

# Optimize for maximal Sharpe ratio
ef = EfficientFrontier(mu, S)
weights = ef.max_sharpe()
cleaned_weights = ef.clean_weights()
ef.portfolio_performance(verbose=True)

# Get the discrete allocation
latest_prices = data.iloc[-1]
da = DiscreteAllocation(weights, latest_prices, total_portfolio_value=10000)
allocation, leftover = da.lp_portfolio()
print(f"Discrete allocation: {allocation}")
print(f"Funds remaining: ${leftover:.2f}")

# Apply optimized weights
optimized_weights = np.array(list(cleaned_weights.values()))
optimized_portfolio_returns = (data.pct_change().dropna() @ optimized_weights)

# Calculate statistics for the optimized portfolio
optimized_stats = {
    "Average Return": np.mean(optimized_portfolio_returns) * 252,
    "Volatility": np.std(optimized_portfolio_returns) * np.sqrt(252),
    "Skewness": skew(optimized_portfolio_returns),
    "Kurtosis": kurtosis(optimized_portfolio_returns),
}

optimized_stats_df = pd.DataFrame(optimized_stats, index=["Optimized Portfolio"])
transposed_optimized_df = optimized_stats_df.T
```

```
109
110 formatted_df = transposed_optimized_df.applymap(lambda x: f"{x:.4f}")
111
112 # Create a figure for the table
113 fig, ax = plt.subplots(figsize=(10, 4))  # Adjust size as needed
114 ax.axis('off')
115
116
117 # Adding the table to the plot
118 table = ax.table(cellText=formatted_df.values,
119                  colLabels=formatted_df.columns,
120                  rowLabels=formatted_df.index,
121                  cellLoc='center',
122                  loc='center')
123
124 table.auto_set_font_size(False)
125 table.set_fontsize(15)  # Adjust as needed
126 table.scale(1, 3)  # Adjust scale factor as needed
127
128 plt.tight_layout()
129 plt.show()
130
131 print(optimized_weights)
```